Практична робота № 5

Тема: Розробка REST-сервісу на базі фреймворку Gin та СУБД PostgreSQL

Мета роботи

- Опанувати принципи розробки високопродуктивних вебсервісів на базі фреймворку Gin (`gin-gonic/gin`) та взаємодію з контекстом запиту `gin.Context`.
- Навчитися застосовувати декларативну валідацію вхідних DTO за допомогою тегів структури `binding` (`go-playground/validator`).
- Опанувати підключення до реляційної СУБД PostgreSQL, проєктування моделей даних та виконання операцій через ORM-бібліотеку GORM.
- Засвоїти криптографічне хешування паролів за допомогою алгоритму `bcrypt` та реалізувати випуск підписаних JWT-токенів (`golang-jwt/jwt/v5`).
- Навчитися захищати приватні маршрути за допомогою кастомного JWT Middleware та генерувати інтерактивну документацію API за допомогою Swagger UI (`swaggo/swag`).

Технічне забезпечення

- Операційна система: Windows, macOS або Linux.
- Середовище розробки: Visual Studio Code (з офіційним розширенням Go) або JetBrains GoLand.
- Компілятор: Go SDK версії 1.22 або новішої.
- Термінал / CLI: bash, zsh або PowerShell.
- Система контролю версій: Git.
- База даних: СУБД PostgreSQL (локально встановлена або запущена через Docker: `docker run -d --name postgres-dev -e POSTGRES_PASSWORD=postgres -p 5432:5432 postgres:alpine`).
- Інструменти тестування: утиліта `curl`, Postman або розширення REST Client / Thunder Client у VS Code.

Короткі теоретичні відомості

1. Архітектура фреймворку Gin та центральна сутність `gin.Context`

Gin — один із найшвидших HTTP-вебфреймворків для Go, побудований навколо спеціалізованого дерева маршрутів на базі префіксного дерева (Radix Tree) та пулу пам'яті контекстів (`sync.Pool`), що мінімізує виділення динамічної пам'яті в купі (heap allocations).

Ключовою точкою входу для будь-якого обробника є вказівник на `*gin.Context`. Він містить усе необхідне для роботи із запитом:

- витягування параметрів шляху: `c.Param("id")`;
- читання query-параметрів: `c.Query("priority")` або `c.DefaultQuery("page", "1")`;
- рендеринг структурованих відповідей: `c.JSON(http.StatusOK, gin.H{"data": result})` або `c.AbortWithStatusJSON(…)`;
- передача даних між Middleware та обробниками: `c.Set("userID", id)` і `c.Get("userID")`.

```
1 r := gin.Default() ①
2
3 v1 := r.Group("/api/v1") ②
4 {
5     v1.GET("/ping", func(c *gin.Context) { ③
6         c.JSON(http.StatusOK, gin.H{"message": "pong"}) ④
7     })
8 }
```

① Ініціалізація двигуна Gin із базовими Middleware логування та відновлення після панік.

② Створення ізольованої групи маршрутів із префіксом `/api/v1`.

③ Реєстрація хендлера з доступом до контексту поточного запиту `*gin.Context`.

④ Формування серіалізованої JSON-відповіді зі статус-кодом 200 OK.

---

2. Декларативна валідація DTO за допомогою тегів `binding`

Замість написання ручних багаторівневих перевірок `if-else`, Gin інтегрує валідатор `go-playground/validator`. Правила перевірки описуються декларативно в тезі `binding`:

```
1 type CreateIncidentDTO struct {
2     Title       string `json:"title" binding:"required,min=5,max=100"` ①
3     Description string `json:"description" binding:"required"`
4     Priority    string `json:"priority" binding:"required,oneof=low medium high
```

```
    critical"` ②
}

func CreateIncidentHandler(c *gin.Context) {
    var dto CreateIncidentDTO
    // ShouldBindJSON десеріалізує тіло запиту та автоматично валідує поля
    if err := c.ShouldBindJSON(&dto); err != nil { ③
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }
    // Дані гарантовано валідні
}
```

① Обов'язкове текстове поле з обмеженням мінімальної та максимальної довжини.

② Обмеження значення поля фіксованим списком допустимих варіантів (`oneof`).

③ Автоматичне зв'язування JSON з викликом перевірки та поверненням 400 у разі невідповідності.

---

3. Робота з PostgreSQL через GORM

Бібліотека **GORM** надає повнофункціональний ORM-шар для Go. Вона бере на себе формування безпечних параметризованих SQL-запитів, автоматичне керування транзакціями та відображення рядків таблиць у структури Go.

```
type Incident struct {
    ID          uint      `gorm:"primaryKey" json:"id"` ①
    Title       string    `gorm:"size:100;not null" json:"title"`
    Description string    `gorm:"type:text;not null" json:"description"`
    Priority    string    `gorm:"size:20;not null" json:"priority"`
    Status      string    `gorm:"size:20;default:'open'" json:"status"`
    CreatedAt   time.Time `json:"created_at"`
}

// Підключення та авто-міграція схеми
dsn := "host=localhost user=postgres password=postgres dbname=postgres port=5432
sslmode=disable"
db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{}) ②
if err != nil {
    log.Fatal(err)
}
db.AutoMigrate(&Incident{}) ③
```

① Визначення первинного ключа та конфігурація колонок таблиці через теги `gorm:"…"`.

② Відкриття пулу з'єднань із базою даних PostgreSQL через офіційний драйвер GORM.

③ Автоматична синхронізація структури Go-моделі зі схемою бази даних (створення/оновлення колонок).

4. Автентифікація JWT (JSON Web Tokens) та хешування `bcrypt`

У клієнт-серверних REST-архітектурах сервер не зберігає стан сесії у пам'яті (Stateless). Замість цього використовується токен JWT, підписаний секретним ключем сервера:

1. **Збереження пароля:** Паролі ніколи не зберігаються у відкритому вигляді. Використовується повільна адаптивна криптографічна функція `bcrypt`:

```
1 hash, err := bcrypt.GenerateFromPassword([]byte(password), bcrypt.DefaultCost) ①
2 // Перевірка:
3 err := bcrypt.CompareHashAndPassword(hash, []byte(password)) ②
```

   ① Отримання безпечного незворотного криптографічного гешу пароля.

   ② Порівняння введеного пароля з існуючим гешем у постійному часі (захист від Timing Attacks).

2. **Генерація JWT:** Після успішної перевірки пароля сервер створює токен, що містить `Claims` (наприклад, ID користувача, строк дії `exp`) та підписує його алгоритмом HMAC-SHA256 (`jwt.SigningMethodHS256`).
3. **JWT Middleware:** Перехоплює заголовок `Authorization: Bearer <token>`, валідує цифровий підпис та дозволяє або блокує (`c.AbortWithStatusJSON(401, …)`) виконання запиту.

Постановка задачі

Сучасні технічні служби підприємств та диспетчерські центри потребують надійного бекенд-сервісу **обліку технічних інцидентів та аварій (Incident Tracker API)**. Сервіс має забезпечувати зберігання історії подій у реляційній базі даних PostgreSQL, гарантувати коректність вхідних даних (пріоритети, непорожні описи), підтримувати систему облікових записів інженерів із захистом приватних операцій через токени JWT та надавати інтерактивну документацію Swagger UI для фронтенд- та мобільних розробників.

Необхідно розробити RESTful вебзастосунок на фреймворку Gin, який реалізує повний цикл роботи з базою даних через GORM, забезпечує реєстрацію та авторизацію користувачів, обмежує доступ до модифікації інцидентів за допомогою JWT Middleware та генерує специфікацію OpenAPI.

Завдання

Практична робота складається з трьох послідовних рівнів. Кожен наступний рівень

базується на попередньому та розширює функціональність розроблюваного рішення.

Рівень 1. Базовий: Створення Gin-застосунку, підключення PostgreSQL та валідація DTO

1. **Ініціалізація проєкту:**
   - Створити окрему робочу директорію проєкту.
   - Ініціалізувати Go-модуль за допомогою `go mod init <назва_модуля>`.
   - Встановити залежності:

```
go get github.com/gin-gonic/gin
go get gorm.io/gorm
go get gorm.io/driver/postgres
```

2. **Налаштування підключення до PostgreSQL:**
   - Розгорнути екземпляр PostgreSQL (локально або через Docker).
   - У пакеті конфігурації/бази даних створити функцію ініціалізації GORM з перевіркою успішного з'єднання.
   - Описати модель `Incident`:
     - `ID` (`uint`, первинний ключ);
     - `Title` (`string`, заголовок інциденту);
     - `Description` (`string`, докладний опис);
     - `Priority` (`string`, рівень важливості: `low`, `medium`, `high`, `critical`);
     - `Status` (`string`, статус вирішення, за замовчуванням `open`);
     - `CreatedAt` (`time.Time`).
   - Виконати авто-міграцію таблиці в базу даних через `db.AutoMigrate(&Incident{})`.
3. **Проєктування DTO та обробників із валідацією:**
   - Створити структуру `CreateIncidentDTO` з тегами валідатора:
     - `Title`: обов'язкове поле, довжина від 3 до 100 символів;
     - `Description`: обов'язкове поле;
     - `Priority`: обов'язкове поле з обмеженням одного з дозволених значень (`oneof=low medium high critical`).
   - Налаштувати групу маршрутів `/api/v1/incidents`.
   - Реалізувати обробник створення `POST /api/v1/incidents`:
     - десеріалізувати тіло через `c.ShouldBindJSON`;
     - у разі помилки валідації повертати статус `400 Bad Request` з детальним переліком некоректних полів;

        - у разі успіху зберігати запис у PostgreSQL та повертати статус `201 Created` із даними створеного інциденту.
    - Реалізувати обробник `GET /api/v1/incidents`:
        - вичитувати всі інциденти з таблиці за допомогою методу `db.Find(&incidents)`;
        - повертати список у форматі JSON зі статусом `200 OK`.
4. **Тестування:**
    - Запустити сервіс та перевірити роботу ендпоінтів через `curl` або Postman.
    - Переконатися, що при передачі невалідного пріоритету (наприклад, `priority: "urgent"`) сервер повертає статус 400 із повідомленням про помилку валідації.

Рівень 2. Ключовий: Повний життєвий цикл інцидентів та автентифікація користувачів (JWT)

1. **Збереження функціональності Рівня 1.**
2. **Завершення CRUD-операцій над інцидентами:**
    - Реалізувати обробник `GET /api/v1/incidents/:id`:
        - витягти параметр `:id` через `c.Param("id")`;
        - знайти запис у базі даних (`db.First(&incident, id)`);
        - у разі відсутності повернути статус `404 Not Found`, у разі успіху — `200 OK`.
    - Реалізувати обробник зміни статусу `PATCH /api/v1/incidents/:id/status`:
        - приймати DTO оновлення статусу з валідацією `binding:"required,oneof=open in_progress resolved closed"`;
        - оновити статус інциденту в PostgreSQL та повернути актуальний стан зі статусом `200 OK`.
    - Реалізувати обробник видалення `DELETE /api/v1/incidents/:id`:
        - видалити запис із бази даних;
        - у разі успіху повернути статус `204 No Content`.
3. **Модель користувача та криптографічне хешування паролів:**
    - Встановити пакети:

```
go get golang.org/x/crypto/bcrypt
go get github.com/golang-jwt/jwt/v5
```

    - Описати модель `User`:
        - `ID` (`uint`, первинний ключ);
        - `Email` (`string`, унікальний індекс `gorm:"uniqueIndex;not null"`);

        - `PasswordHash` (`string`, `json:"-"` — обов'язково приховано від серіалізації);
        - `CreatedAt` (`time.Time`).
    - Виконати авто-міграцію моделі `User`.
4. **Реалізація сервісу автентифікації:**
    - `POST /api/v1/auth/register`:
        - приймати `RegisterDTO` з валідацією (`Email` формату `email`, `Password` мінімум 6 символів);
        - згенерувати геш пароля через `bcrypt.GenerateFromPassword`;
        - зберегти нового користувача в базі даних та повернути статус `201 Created`.
    - `POST /api/v1/auth/login`:
        - приймати облікові дані (`Email`, `Password`);
        - знайти користувача за email у базі даних;
        - перевірити пароль через `bcrypt.CompareHashAndPassword`;
        - при невідповідності повертати статус `401 Unauthorized`;
        - при успіху генерувати підписаний JWT-токен із терміном дії (наприклад, 24 години) та повертати об'єкт `{"token": "<jwt_string>"}` зі статусом `200 OK`.

Рівень 3. Просунутий: Захисний JWT Middleware та інтерактивна документація Swagger UI

1. **Збереження функціональності попередніх рівнів.**
2. **Створення кастомного JWT Auth Middleware:**
    - Написати проміжний обробник `AuthMiddleware(secretKey string) gin.HandlerFunc`:
        - витягувати заголовок `Authorization`;
        - перевіряти наявність префікса `Bearer `;
        - парсити та верифікувати цифровий підпис токена за допомогою секретного ключа;
        - у разі відсутності або недійсності токена негайно переривати запит через `c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "Unauthorized access"})`;
        - при успішній верифікації витягувати `userID` з токена та зберігати його у контексті через `c.Set("userID", claims.UserID)`.
3. **Захист приватних маршрутів:**
    - Організувати групу маршрутів із застосуванням Auth Middleware:
        - публічні маршрути: `POST /auth/register`, `POST /auth/login`, `GET /incidents` (публічний перегляд);

        - захищені маршрути: `POST /incidents` (створення від імені користувача), `PATCH /incidents/:id/status`, `DELETE /incidents/:id`.
    - При створенні інциденту прив'язувати його до `userID` авторизованого інженера.
4. **Автоматична генерація документації Swagger UI:**
    - Встановити CLI-утиліту `swag`:

```
go install github.com/swaggo/swag/cmd/swag@latest
go get github.com/swaggo/gin-swagger
go get github.com/swaggo/files
```

    - Додати декларативні анотації Swag над функцією `main()` (загальна інформація про API, версія, схема безпеки BearerAuth).
    - Описати анотаціями ключові ендпоінти (`@Summary`, `@Tags`, `@Accept`, `@Produce`, `@Param`, `@Success`, `@Failure`, `@Security BearerAuth`).
    - Згенерувати документацію командою `swag init`.
    - Підключити роут документації `r.GET("/swagger/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))`.
    - Відкрити браузер за адресою `http://localhost:8080/swagger/index.html` та успішно протестувати відправку запитів (включаючи авторизацію через кнопку «Authorize»).

Приклад очікуваного виведення програми

Консоль запуску Gin-сервера:

```
[GIN-debug] [WARNING] Creating an Engine instance with the Logger and Recovery
middleware already attached.
[GORM] Connected to PostgreSQL successfully (host=localhost:5432, db=postgres)
[GORM] Auto-migration completed: 'users', 'incidents' tables are up to date.
[GIN-debug] POST   /api/v1/auth/register     --> myapp/handler.RegisterHandler (3
handlers)
[GIN-debug] POST   /api/v1/auth/login        --> myapp/handler.LoginHandler (3
handlers)
[GIN-debug] GET    /api/v1/incidents         --> myapp/handler.ListIncidentsHandler (3
handlers)
[GIN-debug] POST   /api/v1/incidents         --> myapp/handler.CreateIncidentHandler
(4 handlers)
[GIN-debug] PATCH  /api/v1/incidents/:id/status --> myapp/handler.UpdateStatusHandler
(4 handlers)
[GIN-debug] DELETE /api/v1/incidents/:id     --> myapp/handler.DeleteIncidentHandler
(4 handlers)
```

```
[GIN-debug] GET    /swagger/*any             --> gin-swagger.WrapHandler (3 handlers)
[СЕРВЕР] Сервіс успішно запущено на http://localhost:8080
[СЕРВЕР] Swagger UI доступний за адресою: http://localhost:8080/swagger/index.html
```

Приклад запиту авторизації та отримання JWT токена:

```
curl -i -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"engineer@company.internal","password":"SuperSecretPassword123"}'
```

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Tue, 08 Sep 2026 14:10:00 GMT
Content-Length: 154

{
  "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOjEsImV4cCI6MTcyNTgxODIwMH0.d9f8e4a7
b2..."
}
```

Приклад створення інциденту з валідним токеном:

```
curl -i -X POST http://localhost:8080/api/v1/incidents \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." \
  -d '{"title":"Overheating on Gateway-01","description":"Core temperature reached
85C","priority":"critical"}'
```

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Date: Tue, 08 Sep 2026 14:10:25 GMT
Content-Length: 185

{
  "id": 1,
  "title": "Overheating on Gateway-01",
  "description": "Core temperature reached 85C",
  "priority": "critical",
  "status": "open",
  "user_id": 1,
  "created_at": "2026-09-08T14:10:25.123456Z"
```

```
}
```

Приклад блокування без токена або з недійсним токеном:

```
1 curl -i -X DELETE http://localhost:8080/api/v1/incidents/1
```

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8
Date: Tue, 08 Sep 2026 14:10:40 GMT
Content-Length: 39

{
  "error": "Unauthorized access"
}
```

Контрольні питання

1. У чому полягають головні архітектурні відмінності між фреймворком Gin та маршрутизатором Chi?
2. Яким чином сутність `gin.Context` оптимізує навантаження на Garbage Collector (GC) за допомогою пулу об’єктів?
3. Як працює зв’язування даних методом `c.ShouldBindJSON` та чим воно відрізняється від `c.BindJSON` у контексті обробки помилок?
4. Які переваги надає використання ORM-бібліотеки GORM порівняно з прямими запитами через драйвер бази даних `database/sql`?
5. Чому паролі користувачів заборонено гешувати простими геш-функціями (MD5, SHA-256) і чому алгоритм `bcrypt` вважається стандартом безпеки?
6. З яких трьох частин складається структура токена JWT (Header, Payload, Signature) та як сервер перевіряє його цілісність без звернення до БД?
7. Як за допомогою методу `c.AbortWithStatusJSON()` у Middleware перервати виконання подальшого ланцюжка обробників?
8. Який принцип лежить в основі автоматичної генерації документації утилітою `swag` за допомогою спеціальних коментарів у коді?

Критерії оцінювання

Оцінювання здійснюється за шкалою повноти реалізації рівнів практичної роботи:

- **«Задовільно» (Рівень 1):** реалізовано базовий прототип, створено структуру проєкту, програма успішно компілюється та виконує базові сценарії.
- **«Добре» (Рівень 2):** виконано базовий рівень та реалізовано ключову функціональність заняття, забезпечено надійність і безпеку виконання коду.
- **«Відмінно» (Рівень 3):** повністю завершено всі три рівні завдання з дотриманням кращих інженерних практик, чистоти архітектури та оптимізації.
- **Усний захист:** обов'язкове пояснення логіки коду та впевнені відповіді на контрольні питання викладача.

Вимоги до звітності

1. Опублікувати вихідний код у Git-репозиторії.
2. Оформити файл `README.md` з описом проєкту, інструкцією запуску та обов'язковими скриншотами роботи програми (включаючи демонстрацію відсутності помилок).
3. Продемонструвати роботу програми викладачеві та захистити результати.